TECHNICAL REFERENCE & MANUAL

Power BI Architecture, DAX Context & Star Schema

1. Deep Dive: Why RELATED() Fails

The `RELATED()` function in DAX is designed to follow existing active relationships in a Power BI data model to retrieve a single scalar value from a related table. It specifically traverses from the **"Many"** side to the **"One"** side of a relationship (or across 1-to-1 relationships).

Cardinality Breakdown

In your dataset, `WarehouseCosts` contains multiple material breakdown entries for a single **Product ID**:

- **Ghee:** $39
- **Pepper:** $10
- **Rice:** $54
- **Salt:** $24
- **Chicken:** $24
- **Cheese:** $48

Total Aggregate Cost: $39 + $10 + $54 + $24 + $24 + $48 = **$199**

Because **Product ID** is non-unique in `WarehouseCosts`, Power BI cannot establish a 1-to-Many lookup. Executing `RELATED()` fails because the engine cannot pick one scalar value out of six options. Instead, you must aggregate the records using `CALCULATE(SUM(...))`.

2. Complete DAX Function Definitions & Mechanics

FUNCTION / CONSTRUCT	DEFINITION	CORE CONCEPT & MECHANICS
`ADDCOLUMNS()`	An iterator function that creates a new table by taking an existing table and appending calculated columns.	It iterates row-by-row over the input table (`DetailedData`). Inside `ADDCOLUMNS`, a Row Context is active for each row being processed.
`CALCULATE()`	Evaluates an expression inside a modified filter context.	The engine's primary function for context modification. Performs **Context Transition** (converts current Row Context into an equivalent Filter Context) and overrides existing filters.
`VAR / RETURN`	DAX construct used to define local variables within a formula block.	Variables are evaluated once upon declaration. Storing values like `ProductID` prevents recalculation during iteration, increasing DAX evaluation speed.
`SUM()` vs. `SUMX()`	`SUM` adds up numbers in a single column within a filter context. `SUMX` is an iterator evaluating row-by-row.	Use `SUM` inside `CALCULATE` for table filtering, and `SUMX` for dynamic measure execution across visual rows.

3. Data Model Architecture: Fact Tables vs. Dimension Tables

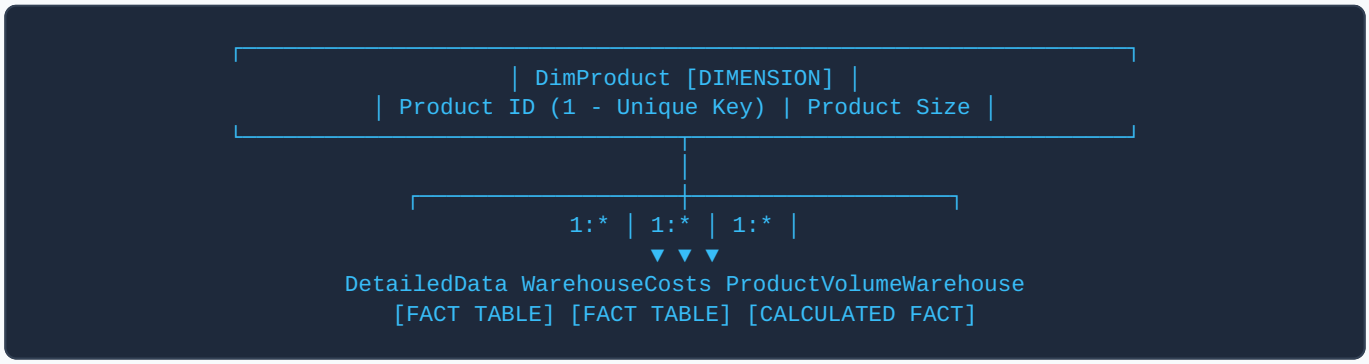
ARCHITECTURAL PILLAR	FACT TABLES DETAILEDDATA WAREHOUSECOSTS	DIMENSION TABLES DIMPRODUCT
Primary Role	Stores numeric business events, transactions, physical shipment quantities, and monetary costs.	Provides master metadata used for slicing, dicing, filtering, and grouping fact records.
Key Attributes	High row count, foreign keys linking to dimensions, duplicate key values, highly additive metrics.	Unique surrogate/business key (1 row per entity), low row count, descriptive text columns (e.g., Size, Category).
In Your Model	`DetailedData` contains unit volume transactions. `WarehouseCosts` contains line-item material cost breakdowns.	`DimProduct` contains unique Product ID entries and master attributes like Product Size.
Filtering Best Practice	**Do NOT place slicers directly on Fact tables.** Fact slicers will fail to filter un-related fact tables.	**ALWAYS place report slicers on Dimension tables.** Filter context flows downward from 1-side to Many-side.

4. Star Schema Architecture & Shared Product Size Slicer

To allow a single Product Size slicer to filter all visuals (whether built from `DetailedData`, `WarehouseCosts`, or calculated tables), establish a Star Schema centered around a shared `DimProduct` table.

Shared Dimension Table Creation

```
-- Create the Shared Dimension Table in Power BI
DimProduct =
DISTINCT(
    SELECTCOLUMNS(
        'DetailedData',
        "Product ID", 'DetailedData'[Product ID],
        "Product Size", 'DetailedData'[Product Size]
    )
)
```



Slicer Execution Mechanics

When a user selects **"Large"** in a report slicer bound to `DimProduct[Product Size]`:

1. Power BI filters `DimProduct` down to only the Product IDs associated with "Large".
2. The active 1-to-Many relationships automatically propagate this filter context downward into `DetailedData`, `WarehouseCosts`, and `ProductVolumeWarehouse`.
3. All visuals on the report page update simultaneously using a single common control.

5. Recommended DAX Calculated Table Code

```
ProductVolumeWarehouse =
ADDCOLUMNS(
    'DetailedData',
    "Warehouse Cost",
        VAR ProductID = 'DetailedData'[Product ID]
        RETURN
            CALCULATE(
                SUM('WarehouseCosts'[Cost at warehouse in USD]),
                'WarehouseCosts'[Product Id] = ProductID
            ),
    "Volume × Warehouse Cost",
        VAR ProductID = 'DetailedData'[Product ID]
        VAR Volume = 'DetailedData'[Volume of material to be shippped in units]
        VAR WarehouseCost =
            CALCULATE(
                SUM('WarehouseCosts'[Cost at warehouse in USD]),
                'WarehouseCosts'[Product Id] = ProductID
            )
        RETURN
            Volume * WarehouseCost
)
```

6. Critical Architecture Rule: Calculated Tables vs. DAX Measures

FEATURE / DIMENSION	CALCULATED TABLES `ADDCOLUMNS`	DAX MEASURES `SUMX`
Evaluation Timing	Computed **ONLY during data refresh**. Stored physically in RAM.	Computed **ON-THE-FLY** at visual rendering time.
Slicer Responsiveness	Static row data does **NOT** recompute dynamically when report slicers change.	**Dynamic!** Fully recalculates based on user slicer selections in real-time.
Best Use Case	Creating dimensions, structural data modeling, bridge tables.	Calculating KPIs, totals, revenues, multi-fact volume calculations.

Dynamic Measure Alternative (Best Practice for Enterprise Power BI)

```
-- Dynamic Measure Alternative (Best Practice for Enterprise Power BI)
Total Volume × Warehouse Cost =
```

```
SUMX(
    'DetailedData',
    VAR ProductID = 'DetailedData'[Product ID]
    VAR Volume = 'DetailedData'[Volume of material to be shippped in units]
    VAR WarehouseCost =
        CALCULATE(
            SUM('WarehouseCosts'[Cost at warehouse in USD]),
            'WarehouseCosts'[Product Id] = ProductID
        )
    RETURN
        Volume * WarehouseCost
)
```